

Anexo A

Comandos habituales

Este capítulo presenta brevemente algunos de los programas más comunes y útiles cuando se trabaja con la shell. Salvo que se indique otro paquete, todos ellos pertenecen a GNU (coreutils).

A.1. Ficheros y directorios

cp (*copy*)

dados un nombre de archivo existente y un directorio o nombre de archivo, copia el primero en el segundo. Si el archivo destino existe, lo sobrescribe.

cut escribe en su salida partes de las líneas de entrada según sus opciones:

```
alice@maine:~$ date
Sun Aug 26 12:47:35 CEST 2012
alice@maine:~$ date | cut --delimiter=" " --fields=2
Aug
```

diff ()

muestra las diferencias entre dos archivos.

find ()

encuentra archivos a partir de su nombre.

head escribe a su salida los primeros bytes o líneas de su entrada o del archivo indicado como argumento.

```
alice@maine:~$ head --lines=3 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
```

ln (*link*)

crea enlaces a otros archivos o directorios.

md5sum

calcula (o comprueba) la suma MD5 del archivo indicado (o de su entrada estándar).

mkdir (*make dir*)

crea un directorio.

mkfifo

crea una tubería con nombre ligado al sistema de archivos.

mv (*move*)

es equivalente a **cp** pero borra el archivo original.

nl (*number lines*)

lee líneas de un archivo y las escribe a su salida precedidas del número de línea.

pwd (*print working directory*)

indica el nombre del directorio actual.

rm (*remove*)

elimina el archivo indicado.

rmdir (*remove dir*)

borra un directorio vacío.

seq (*sequence*)

escribe un rango de enteros. Puede usarse como variable de control de un bucle **for**.

split

divide un archivo en trozos del tamaño indicado.

stat produce información detallada de archivos.

tac (*reverse cat*)

escribe a su salida líneas de su entrada en orden inverso.

tail escribe a su salida los últimos bytes o líneas de su entrada o del archivo indicado como argumento. Con la opción **-f/--follow** monitoriza el archivo mostrando el nuevo contenido tan pronto como se añade. Muy útil para hacer el seguimiento de un archivo de log.

tee crea una «te». Lee de su entrada estándar y lo escribe al mismo tiempo a su salida estándar y al archivo indicado.

test, [¹

realiza comprobaciones lógicas sobre archivos, cadenas de texto y datos numéricos. Se utiliza habitualmente como condición en estructuras de control **if**, **while**, etc.

touch

cambia la fecha de un archivo (por defecto ahora). Si el archivo indicado no existe, crea uno vacío.

¹No es una errata: [, el corchete de apertura

tr (*translate*)

lee líneas de su entrada y las escribe a su salida reemplazando *tokens*² tal como lo indiquen sus opciones.

uniq lee líneas de su entrada y las escribe a su salida, pero omitiendo líneas consecutivas idénticas.

wc (*word count*)

cuenta letras, palabras y líneas del archivo indicado (o de su entrada estándar) y escribe los totales en su salida.

yes escribe «y» y un salto de línea continuamente a su salida. Se utiliza para contestar afirmativamente a cualquier pregunta que haga un programa por línea de comando:

```
alice@maine:~$ touch kk
alice@maine:~$ yes | rm --interactive --verbose kk
rm: remove regular empty file `kk'? removed `kk'
```

A.2. Sistema

dd copia bloques de bytes en dispositivos (archivos o discos).

df (*disk free*) muestra información sobre el uso de los sistemas de archivos montados en el computador.

du (*disk usage*) calcula el espacio de disco utilizado por archivos y directorios.

fdisk

muestra y manipula tablas de particiones.

mount

monta dispositivos de almacenamiento de cualquier tipo sobre el sistema de archivos.

uname

(*unix name*) muestra información del sistema: núcleo, hostname, versión, arquitectura, etc.

```
alice@maine:~$ uname -a
Linux maine 6.12.38+deb13-amd64 #1 SMP Debian 6.12.38-1 (2025-07-16) x86_64
↳ GNU/Linux
```

sync escribe a disco inmediatamente las operaciones pendientes sobre archivos.

²Un *token* es cualquier combinación de caracteres que cumpla una expresión regular concreta.

A.3. Procesos

nice ejecuta un programa fijando un nivel de prioridad.

nohup

ejecuta un comando ignorando las señales para su finalización (SIGHUP).
Permite que un proceso creado por una shell sobreviva a la propia shell.

sleep

realiza una pausa del número de segundos indicados.

true, false

simplemente retornan 0 y 1, los valores que corresponden a una ejecución correcta e incorrecta respectivamente.

top (🖨️ **procps**)

, muestra una lista actualizada de los procesos ordenada por el consumo de CPU.

A.4. Usuarios y permisos

chmod (*change mode*)

cambia los permisos de lectura, escritura y ejecución de un archivo o directorio.

chown (*change owner*)

cambia el propietario (usuario y grupo) de un archivo.

chgrp (*change group*)

cambia el grupo propietario de un archivo.

groups

muestra los nombres de los grupos a los que pertenece el usuario.

id

muestra los identificadores del usuario y los grupos a los que pertenece.

sudo

(*superuser do*) permite ejecutar un comando con los privilegios de otro usuario, por defecto el administrador.

who

muestra los usuarios conectados junto con los terminales que utilizan y la hora de la conexión (*login*).

whoami

muestra el nombre del usuario conectado.